OBJECT-ORIENTED PROGRAMMING · NRC 36862

LegalSync

Legal Case Management System

Team
CodeLex

Instructor
PhD. Jorge Edison Lascano

Institution
Universidad de las Fuerzas Armadas ESPE

Ostin Christopher Torres · Josselyn Ulcuango · Mateo Unda · Yandry Vélez

First Partial · October 2026

CURRENT CHALLENGE

The Problem: A Law Firm Operating on Excel

Cases, billing and hearings are tracked in manual spreadsheets. Six different staff roles touch the same records with no credentials, no history and no way to know how much each client owes.

⏱

Missed Deadlines

No structured calendar or task log for hearings, filings and pending work.

📁

Scattered Data

Documents and evidence live apart from the case file; no quick multi-criteria search.

👤

No Traceability

Impossible to tell which user modified a case or a payment record, and when.

$

Unclear Client Debt

Court fees, notary copies and other costs are not itemized per case, so balances are guesswork.

Roles involved: Managing Director · Partner · Associate Lawyer · Junior Lawyer · Legal Assistant · Secretary — all with the same unrestricted access today.

PROJECT DEFINITION (V2.0)

Objective and Scope

MAIN OBJECTIVE

Centralize law-firm case management in a Java OOP console application

A keyboard-driven menu that registers clients and classified case files, controls access through a 6-role hierarchy identified by email, tracks expenses and client debt, and logs every change.

- Hierarchical, email-based roles managed only by the **DIRECTOR**
- Cases classified by **category, priority and status** (ACTIVE / PASSIVE ...)
- Per-case **expense ledger** with automatic debt: PENDING or PAID IN FULL
- Tasks, hearings and calendar with **remaining days**; document file paths
- **JSON** persistence, **CSV** reports and an **audit log** of every action

First-partial scope: NetBeans console application + unit tests. GUI and database are planned for later units.

REQUIREMENTS TRACEABILITY

Product Backlog: All 9 Requirements Implemented

ID	Requirement	Where it lives in the code
PB-001	Role & access management by email	`LegalSyncService.registerOrUpdateUserRole()`, `User.hasPermission()`
PB-002	Login & credential validation	`LegalSyncService.authenticate()`, `User.login()`, `changePassword()`
PB-003	Register clients and defendants	`registerClient()`, `Client.register()`, `Defendant.update()`
PB-004	Register, code & classify cases	`registerCaseFile()`, `CaseFile.validateMandatoryFields()`, `updateCaseStatus()`
PB-005	Multi-criteria search	`findCaseById()`, `searchByClientName()`, `filterByCategory()`, `filterByStatus()`
PB-006	Expense ledger & client debt	`addExpenseToCase()`, `markExpenseAsPaid()`, `CaseFile.calculateTotalDebt()`
PB-007	Dispositions, tasks & calendar	`addDeadlineToCase()`, `completeDeadlineTask()`, `Deadline.calculateRemainingDays()`
PB-008	Document paths & audit log	`attachDocumentToCase()`, `recordAudit()` → `data/audit_log.json`
PB-009	CSV report generation	`FileManager.generateCsvReports()` → `cases_report.csv`, `billing_report.csv`

SOFTWARE ARCHITECTURE

Project Structure: Layered Packages

Package root `ec.edu.espe.legalsync` — each layer has one responsibility.

view

LegalSyncSystem

main() and the 0–9 console menu, input reading with Scanner.

service

LegalSyncService

Business logic: checks permissions, runs operations, saves, audits.

model

11 domain classes

User, Lawyer, LegalAssistant, Secretary, Client, Defendant, CaseFile, Document, Deadline, Expense, AuditEntry.

utils

FileManager

Static methods to read/write JSON files and export CSV reports.

exception

LegalSyncException

Custom checked exception with an error code.

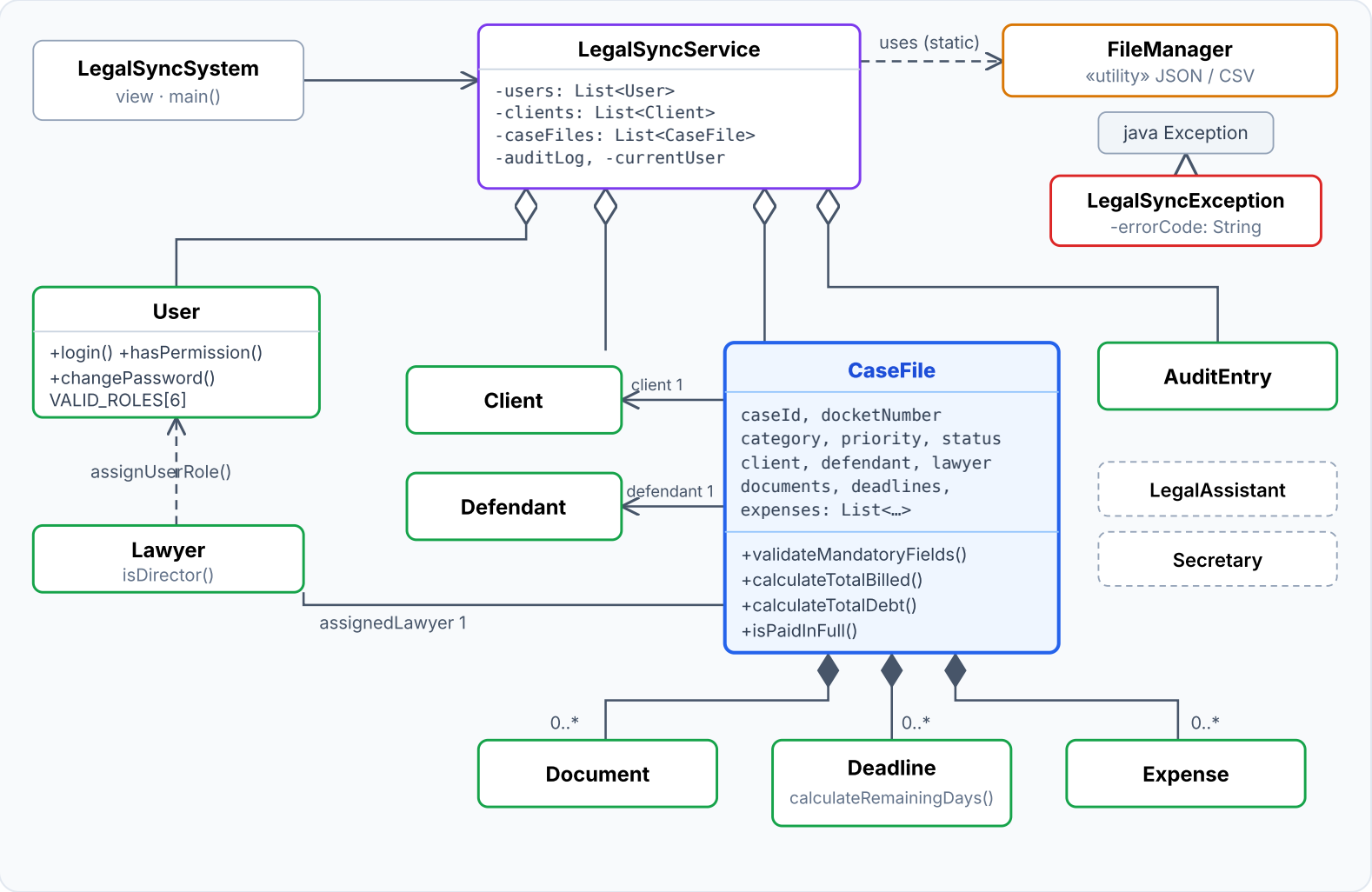
view → service → model + utils → `data/*.json` · `reports/*.csv`

NetBeans Ant project · JDK 24

Source: 15 classes in 5 packages | Test packages: 13 JUnit 4 test classes, 219 tests

OBJECT-ORIENTED DESIGN

Class Diagram as Implemented



15 classes · 5 packages

11 model classes plus service, utils, view and exception.

CaseFile is the core entity

Links client, defendant, lawyer, documents, deadlines and expenses.

Legend

◆ composition ◇ aggregation

→ association ⇢ dependency

△ inheritance

Dashed boxes: staff profiles stored as standalone classes.

CLASS ARCHITECTURE

Class Relationships in Code

Association

Independent objects

A `CaseFile` references its `Client`, `Defendant` and assigned `Lawyer`. The client exists before the case and can have several cases.

Composition

Lifecycle bound to the case

The `CaseFile()` constructor creates its own `documents`, `deadlines` and `expenses` lists; they only exist inside that case.

Aggregation

Shared containers

`LegalSyncService` holds `List<User>`, `List<Client>`, `List<CaseFile>` and `List<AuditEntry>` loaded from JSON.

Dependency

Temporary use

`Lawyer.assignUserRole(User, role)` receives a user as a parameter; the service calls the static `FileManager` methods without storing them.

Inheritance

"is-a"

`LegalSyncException extends Exception`: adds an `errorCode` and reuses the Java exception mechanism with `throw` / `catch`.

Polymorphism

Overriding

Every model class overrides `toString()`, and the menu catches any `LegalSyncException` through its parent type.

OOP PRINCIPLES

Encapsulation and Abstraction in CaseFile

```java
public class CaseFile {
    public static final String[] VALID_STATUSES =
        {"ACTIVE", "PASSIVE", "IN_PROGRESS",
         "SUSPENDED", "CLOSED"};
    private String status;
    private List<Expense> expenses;

    public void setStatus(String status) {
        if (isValidValue(status, VALID_STATUSES)) {
            this.status = status.toUpperCase();
        }   // invalid values are ignored
    }

    public double calculateTotalDebt() {
        double debt = 0.0;
        for (Expense exp : expenses)
            if (!exp.isPaid()) debt += exp.getAmount();
        return debt;
    }
}
```

Encapsulation

Private attributes. Setters validate: `setStatus()` rejects unknown states and `Expense.setAmount()` rejects negative amounts.

Abstraction

Callers ask the case "how much is owed?" without knowing how expenses are stored.

Safe defaults

The empty constructor starts every case as ACTIVE · MEDIUM · CIVIL with empty lists.

SYLLABUS APPLICATION

Unit 1 Core Topics Applied in LegalSync

Syllabus topic	Implementation example in LegalSync
Classes & objects	11 domain classes instantiated from JSON data and console input
Encapsulation	Private attributes; password changes only through `User.changePassword(old, new)`
Constructors	Empty + full constructors; `CaseFile()` initializes lists and default status, priority and category
Getters & setters	Validating setters: `setRole()`, `setStatus()`, `setPriority()`, `setAmount()`
Constant arrays	`VALID_ROLES`, `VALID_CATEGORIES`, `VALID_PRIORITIES`, `VALID_STATUSES`
Collections	`List` / `ArrayList` of users, clients, cases, expenses, deadlines and documents
File I/O	`BufferedReader` / `PrintWriter` for JSON files and CSV reports; `Scanner` for keyboard input
Static methods	`User.isValidRole()`, `CaseFile.isValidValue()`, all `FileManager` methods
Exceptions	Custom `LegalSyncException` + `try/catch` for `NumberFormatException` and date parsing

SECURITY CONTROL · PB-001 / PB-002

Roles & Permissions Matrix

Operation	Director	Partner	Associate	Junior	Assistant	Secretary
Login, change own password	✓	✓	✓	✓	✓	✓
View / search cases, calendar, debts	✓	✓	✓	✓	✓	✓
CSV reports and audit log	✓	✓	✓	✓	✓	✓
Register / update clients, cases, expenses, tasks, documents	✓	✓	✓	✓	✓	✗
Create users, assign roles, enable / disable accounts	✓	✗	✗	✗	✗	✗

```
if ("MANAGE_USERS".equalsIgnoreCase(op)) return "DIRECTOR".equalsIgnoreCase(role);
if ("WRITE".equalsIgnoreCase(op))        return !"SECRETARY".equalsIgnoreCase(role);
return true;  // READ allowed for every active role      — User.hasPermission()
```

EXECUTION FLOW

System Workflow and Debt Statement (PB-005, PB-006)

1. Load JSON data (or create default data)
2. Login with registered email and password
3. Menu 0–9, one option per PB-001...PB-009
4. Service checks permission, runs the operation and saves to JSON
5. Every change is written to `audit_log.json`

```
Select an operation (0-9): 5  →  Search by Case ID: 101
 CASE DOCKET DETAILS: 17203-2026-00451 (Case ID: 101)
 Title         : Alimony Claim Gomez vs Perez
 Process Class : Summary Proceeding | Category: FAMILY | Priority: HIGH
 Judicial Unit : Family Judicial Unit North Quito | Status: ACTIVE
 CLIENT PROFILE: Maria Fernanda Gomez | ID/RUC: 1712345678

 --- ITEMIZED EXPENSE & CLIENT DEBT LEDGER ---
  Expense 1 | 2026-10-12 | $5.00   | PENDING | Notarized copies
  Expense 2 | 2026-10-15 | $15.00  | PENDING | Courier notification fee
  Expense 3 | 2026-10-10 | $50.00  | PAID    | Initial consultation fee
 TOTAL BILLED: $70.00 | PENDING CLIENT DEBT: $20.00
 ACCOUNT STATUS: PENDING BALANCE (Debe)

Op 6 → mark Expense 1 as PAID  →  pending debt drops to $15.00
```

ROBUSTNESS

File Persistence & Exception Handling

Storage: `data/` and `reports/`

`users.json`
Accounts, roles and active flag

`clients.json`
Client profiles

`cases.json`
Cases with expenses, deadlines and documents

`audit_log.json`
Date, user, role, action and docket of every change

`cases_report.csv · billing_report.csv`
Exported reports for the Director and the Secretary

LegalSyncException error codes

`ERR-AUTH · ERR-CREDENTIALS · ERR-DISABLED`
Not logged in, wrong password, account disabled

`ERR-ROLE-READONLY · ERR-DIRECTOR-ONLY`
Secretary writing data; non-director managing users

`ERR-CASE-FIELDS · ERR-DUPLICATE-ID · ERR-AMOUNT`
Missing mandatory fields, repeated case ID, amount ≤ $0

`NumberFormatException`
readInt() / readDouble() ask again instead of crashing

`Invalid dates · missing files`
Remaining days returns 0; missing JSON loads empty lists

QUALITY ASSURANCE

JUnit 4 Unit Testing Results

51.14%

112 of 219 tests passing

13 test classes · model, service and utils

Why tests fail

- Generated expected values (`""`, `0`) vs. real results (`null`, formatted text)
- Service methods called with no user logged in → **ERR-AUTH**
- FileManager save methods receiving `null` lists

Test class	Passed	Pass rate
ExpenseTest	11 / 15	73.3%
CaseFileTest	26 / 40	65.0%
DeadlineTest	13 / 20	65.0%
LawyerTest	9 / 15	60.0%
UserTest	10 / 18	55.6%
DocumentTest	7 / 13	53.8%
Client · Defendant · LegalAssistant · Secretary	25 / 50	50.0%
AuditEntryTest	5 / 12	41.7%
LegalSyncServiceTest	5 / 26	19.2%
FileManagerTest	1 / 10	10.0%

ITERATIVE DESIGN

Evolution: From Version 1.0 to Version 2.0

After the client (customer) meeting on October 17, 2026, the definition and the backlog were updated and the code follows version 2.0.

Change	Rationale
TXT / CSV storage → JSON files	Structured persistence for users, clients, cases and audit log
6-role hierarchy identified by email	Only the DIRECTOR creates accounts, assigns roles and disables access
Case classification	Category (Criminal, Family, Civil, Labor, Transit), priority and ACTIVE / PASSIVE status
Expense ledger & client debt	Itemized costs per case with automatic PENDING / PAID IN FULL status
CSV report export	`cases_report.csv` and `billing_report.csv` for the Director and Secretary
Custom LegalSyncException	One exception type with error codes instead of scattered error messages
Menu aligned to the backlog	9 operations, each one labelled with its PB-00X requirement

SUMMARY

Conclusions

Solid OOP design

Encapsulation, composition, aggregation, association and inheritance applied to a real law-firm problem.

Data integrity & security

Role-based access, validated input, JSON persistence and a full audit trail.

Quality assurance

219 unit tests define the next refinements: expected values, logged-in test context and null handling.

Next steps: graphical interface, password hashing, a common `Staff` superclass for staff profiles, and database integration.

Questions?

LegalSync · Team CodeLex

Demo credentials

`director@legalsync.com / dRtor26`
`secretary@legalsync.com / seC32`